Ur Sandhai

AI for Bharat, Prototype Round Write-Up — Track: AgriTech — Individual entry

Live demo: https://sruthisureshkumar-arch.github.io/ursandhai/

Source code: https://github.com/sruthisureshkumar-arch/ursandhai

---

What This Prototype Demonstrates

Ur Sandhai lets farmers within about ten kilometres of each other share what they're selling, at what price, and how strong the demand feels, then turns those reports into a same-day recommendation: the best time to sell, the crop worth prioritising, and whether to hold back because too many neighbours are already offering the same thing. The recommendation is spoken and shown in both Tamil and English. This build is a single self-contained HTML file with no backend and no build step, by design, so it can be opened and tested in a few seconds without installing anything.

What's New and Novel in This Round

Three mechanics go beyond a simple average-price lookup, aimed directly at the moment a farmer is deciding whether to sell. Fair Offer Check lets a farmer standing in front of a buyer type in the price being offered and get an instant verdict against the live local average, in English and Tamil, turning the project's opening problem statement into a usable tool rather than a paragraph. The Sell Together nudge watches for real oversupply among nearby crowd reports and suggests coordinating a shared sale, turning a penalty the scoring model already computes into a positive, actionable feature. The price trend line is a genuine ordinary-least-squares regression, run in the browser on whatever reports currently exist for a crop, showing whether the local price is rising, falling, or steady with a percent-per-day figure, a lightweight but real stand-in for the Prophet model planned for production.

What's Actually Working Right Now

The geo-radius filtering is a real Haversine distance calculation, written in plain JavaScript, that keeps each report to a ten-kilometre radius and a seventy-two hour window. Prices are bucketed into two-hour windows and averaged to surface which hours are currently fetching the best prices. A weighted scoring formula combines average price, demand strength, and a supply-pressure penalty, so the app accounts for oversupply nearby rather than just recommending whichever crop happens to be selling for the most. The ten kilometre radius, the farmer's position, and every nearby report are rendered as a real vector map generated from the same coordinate math the scoring logic uses, not a static image. Every recommendation is produced in both English and Tamil from the same underlying data, and when fewer than five reports exist nearby, the app falls back to a baseline mandi price rather than guessing from thin data. Photographing a price board now runs real optical character recognition: the image is passed to Tesseract.js, a free open-source OCR engine, entirely inside the browser, and a recognised price is read straight into the form.

What's Mocked for This Round, and Why

Voice input and spoken output are stood in with free browser APIs, since wiring up billed cloud services for a prototype round didn't seem like the right place to spend the team's limited AWS free-tier hours. Voice input uses the browser's Web Speech API instead of Amazon Transcribe, listening in Tamil and transcribing locally, with the same regex-based parsing that would run against Transcribe's output extracting a price from the transcript. Spoken recommendations use the Web Speech Synthesis API instead of Amazon Polly, reading the Tamil recommendation aloud with a Tamil voice. Everything downstream of those two inputs, meaning the scoring, the map, the fair-offer check, the sell-together nudge, the trend regression, and the bilingual text generation, is real and does not depend on any external service. One piece from the original proposal, identifying a crop from a photo of the produce itself as opposed to reading text off a price board, which now works, is not wired into this build yet and is scoped as the next milestone.

Tools, APIs, and Datasets

Layer	Used now	Planned for production
Frontend	Plain HTML, CSS, and JavaScript, no framework or build tool	Same, or a lightweight framework if report volume grows
Design	Figma, used to prototype the visual language before implementing it in code	Same
Voice input	Web Speech API (SpeechRecognition), free and browser-native	Amazon Transcribe (Tamil and Hindi) within the AWS free tier
Text-to-speech	Web Speech Synthesis API, free and browser-native	Amazon Polly within the AWS free tier
OCR (price boards)	Tesseract.js, free and open-source, runs fully in the browser	Amazon Textract within the AWS free tier

Vision (crop ID)	Not yet implemented	Amazon Rekognition
Forecasting	A real OLS linear regression on crowd reports, computed client-side	Prophet and scikit-learn, retrained nightly on crowd reports via a free GitHub Actions workflow
Government dataset	A small hard-coded baseline standing in for the real feed	Agmarknet mandi prices and arrivals, pulled from data.gov.in at no cost
Hosting	GitHub Pages, free static hosting	Same for the demo, plus a small backend once reports need to persist across visitors
Data storage	Browser localStorage, so a demo session survives a page refresh	Supabase Postgres free tier, chosen for built-in geo queries
Fonts	Playfair Display and Inter, served from Google Fonts	Same

Feasibility, Cost, and a Path to Scale

Every piece of this prototype runs on free tiers with no ongoing bill: GitHub Pages for hosting, free CDN fonts and libraries, and browser-native speech APIs. Nothing here can generate a surprise invoice because nothing paid is wired in yet. The production plan carries the same discipline forward rather than abandoning it once real cloud services are added: AWS credentials would never touch the client, every AI call would run server-side behind authentication and rate limits, IAM roles would be scoped to a single service each, and a hard budget alarm would cut access automatically if spending ever moved past zero. A realistic rollout starts with a handful of villages in one district, using the crowd-report flow and the Agmarknet baseline as the cold-start fallback, then expands district by district as each area builds up enough regular reporters to keep the crowd data fresh. The intent is for cost per additional farmer to stay close to zero throughout, since the architecture is designed to be serverless and free-tier, but that is a design goal for a system that has not been built or tested at any real scale, not a measured outcome.

Social Impact

A farmer who doesn't know the fair price for their crop that day, in their own village, is at a structural disadvantage against a buyer who does. A five to ten percent improvement in the timing of a sale, or in catching a lowball offer before agreeing to it, is a reasonable illustrative estimate of what that could be worth given typical day-to-day mandi price swings, not a measured result; this prototype has not been piloted with real farmers, so there is no outcome data behind that figure yet. The Sell Together nudge is a narrower, more honest claim: it surfaces a suggestion to coordinate a shared sale when it detects real oversupply nearby. It does not yet give farmers a way to actually contact each other, so today it is a heads-up rather than a working coordination tool the way an organised cooperative would be.

Honest Note on Scope

This is a working demonstration of the core idea, hyperlocal crowd data turned into a same-day, bilingual selling recommendation, not a finished production system. The parts that are mocked are mocked with free, zero-setup browser APIs specifically so the full flow can be tested without needing API keys or a backend. The GitHub repository's README carries the same real-versus-mocked breakdown alongside instructions for running the prototype locally.